

WEEK 12 PRACTICAL

QUESTION-1: Implement the Producer-Consumer problem using counting semaphores and POSIX threads. Evaluate synchronization correctness and system throughput under different buffer sizes.

AIM

To implement the Producer-Consumer problem using POSIX threads and counting semaphores, and compare throughput for different buffer sizes.

PROGRAM

```

#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <time.h>

#define ITEMS 100000

int *buffer;
int buffer_size;
int in = 0, out = 0;

sem_t empty_slots;
sem_t full_slots;
pthread_mutex_t mutex;

long long produced_sum = 0;
long long consumed_sum = 0;

void *producer(void *arg)
{
    for (int i = 1; i <= ITEMS; i++) {
        sem_wait(&empty_slots);

        pthread_mutex_lock(&mutex);

        buffer[in] = i;
        in = (in + 1) % buffer_size;
        produced_sum += i;

        pthread_mutex_unlock(&mutex);

        sem_post(&full_slots);
    }

    return NULL;
}

void *consumer(void *arg)
{
    for (int i = 1; i <= ITEMS; i++) {
        sem_wait(&full_slots);

        pthread_mutex_lock(&mutex);

        int item = buffer[out];
        out = (out + 1) % buffer_size;
        consumed_sum += item;
    }
}

```

```

        pthread_mutex_unlock(&mutex);

        sem_post(&empty_slots);
    }

    return NULL;
}

int main(int argc, char *argv[])
{
    if (argc != 2) {
        printf("Usage: %s <buffer_size>\n", argv[0]);
        return 1;
    }

    buffer_size = atoi(argv[1]);

    if (buffer_size <= 0) {
        printf("Buffer size must be greater than 0.\n");
        return 1;
    }

    buffer = malloc(buffer_size * sizeof(int));

    if (buffer == NULL) {
        printf("Memory allocation failed.\n");
        return 1;
    }

    sem_init(&empty_slots, 0, buffer_size);
    sem_init(&full_slots, 0, 0);
    pthread_mutex_init(&mutex, NULL);

    pthread_t producer_thread, consumer_thread;

    struct timespec start, end;

    clock_gettime(CLOCK_MONOTONIC, &start);

    pthread_create(&producer_thread, NULL, producer, NULL);
    pthread_create(&consumer_thread, NULL, consumer, NULL);

    pthread_join(producer_thread, NULL);
    pthread_join(consumer_thread, NULL);

    clock_gettime(CLOCK_MONOTONIC, &end);

    double time_taken =

```

```

        printf("Memory allocation failed.\n");
        return 1;
    }

    sem_init(&empty_slots, 0, buffer_size);
    sem_init(&full_slots, 0, 0);
    pthread_mutex_init(&mutex, NULL);

    pthread_t producer_thread, consumer_thread;

    struct timespec start, end;

    clock_gettime(CLOCK_MONOTONIC, &start);

    pthread_create(&producer_thread, NULL, producer, NULL);
    pthread_create(&consumer_thread, NULL, consumer, NULL);

    pthread_join(producer_thread, NULL);
    pthread_join(consumer_thread, NULL);

    clock_gettime(CLOCK_MONOTONIC, &end);

    double time_taken =
        (end.tv_sec - start.tv_sec) +
        (end.tv_nsec - start.tv_nsec) / 1e9;

    double throughput = ITEMS / time_taken;

    printf("\nBuffer Size    : %d\n", buffer_size);
    printf("Items              : %d\n", ITEMS);
    printf("Produced Sum       : %lld\n", produced_sum);
    printf("Consumed Sum       : %lld\n", consumed_sum);
    printf("Time                : %.6f seconds\n", time_taken);
    printf("Throughput         : %.2f items/second\n", throughput);

    if (produced_sum == consumed_sum)
        printf("Synchronization: CORRECT\n");
    else
        printf("Synchronization: ERROR\n");

    sem_destroy(&empty_slots);
    sem_destroy(&full_slots);
    pthread_mutex_destroy(&mutex);
    free(buffer);

    return 0;
}

```

OUTPUT

```
ashborn@DESKTOP-I4G5Q4N:~/week12_p$ for size in 2 5 10; do ./q1 $size; done

Buffer Size   : 2
Items         : 100000
Produced Sum  : 5000050000
Consumed Sum  : 5000050000
Time          : 1.309833 seconds
Throughput    : 76345.60 items/second
Synchronization: CORRECT

Buffer Size   : 5
Items         : 100000
Produced Sum  : 5000050000
Consumed Sum  : 5000050000
Time          : 0.260853 seconds
Throughput    : 383358.12 items/second
Synchronization: CORRECT

Buffer Size   : 10
Items         : 100000
Produced Sum  : 5000050000
Consumed Sum  : 5000050000
Time          : 0.186743 seconds
Throughput    : 535495.59 items/second
Synchronization: CORRECT
```

OBSERVATION

Counting semaphores `empty_slots` and `full_slots` control the available and filled buffer positions, while the mutex protects the shared buffer. The equal produced and consumed sums confirm correct synchronization. Changing the buffer size affects the time taken and system throughput.

RESULT

The Producer-Consumer problem was successfully implemented using POSIX threads, counting semaphores, and a mutex. Synchronization correctness and throughput were evaluated using different buffer sizes.

QUESTION-2: Design a deadlock scenario involving multiple threads competing for shared resources. Demonstrate deadlock occurrence, identify the four necessary conditions for deadlock, and implement a deadlock prevention strategy using resource ordering.

AIM

To demonstrate a deadlock between multiple threads and prevent it by enforcing a fixed resource-ordering strategy.

PROGRAM

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t resource1 = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t resource2 = PTHREAD_MUTEX_INITIALIZER;

/* Deadlock version */
void *thread1_deadlock(void *arg)
{
    pthread_mutex_lock(&resource1);
    printf("Thread 1 locked Resource 1\n");

    sleep(1);

    pthread_mutex_lock(&resource2);
    printf("Thread 1 locked Resource 2\n");

    pthread_mutex_unlock(&resource2);
    pthread_mutex_unlock(&resource1);

    return NULL;
}

void *thread2_deadlock(void *arg)
{
    pthread_mutex_lock(&resource2);
    printf("Thread 2 locked Resource 2\n");

    sleep(1);

    pthread_mutex_lock(&resource1);
    printf("Thread 2 locked Resource 1\n");

    pthread_mutex_unlock(&resource1);
    pthread_mutex_unlock(&resource2);

    return NULL;
}

/* Deadlock prevention using resource ordering */
void *thread1_safe(void *arg)
{
    pthread_mutex_lock(&resource1);
    printf("Thread 1 locked Resource 1\n");

    sleep(1);
```

```

    sleep(1);

    pthread_mutex_lock(&resource2);
    printf("Thread 1 locked Resource 2\n");

    printf("Thread 1 completed safely\n");

    pthread_mutex_unlock(&resource2);
    pthread_mutex_unlock(&resource1);

    return NULL;
}

void *thread2_safe(void *arg)
{
    /* Same order: Resource 1 -> Resource 2 */
    pthread_mutex_lock(&resource1);
    printf("Thread 2 locked Resource 1\n");

    sleep(1);

    pthread_mutex_lock(&resource2);
    printf("Thread 2 locked Resource 2\n");

    printf("Thread 2 completed safely\n");

    pthread_mutex_unlock(&resource2);
    pthread_mutex_unlock(&resource1);

    return NULL;
}

int main()
{
    pthread_t t1, t2;

    printf("=== Deadlock Prevention using Resource Ordering ===\n\n");

    pthread_create(&t1, NULL, thread1_safe, NULL);
    pthread_create(&t2, NULL, thread2_safe, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    printf("\nDeadlock avoided by acquiring resources in a fixed order.\n");

    return 0;
}

```

OUTPUT

```
ashborn@DESKTOP-I4G5Q4N:~/week12_p$ ./q2
=== Deadlock Prevention using Resource Ordering ===

Thread 1 locked Resource 1
Thread 1 locked Resource 2
Thread 1 completed safely
Thread 2 locked Resource 1
Thread 2 locked Resource 2
Thread 2 completed safely

Deadlock avoided by acquiring resources in a fixed order.
```

OBSERVATION

Deadlock can occur when threads hold one resource while waiting for another resource held by another thread. The four necessary conditions are mutual exclusion, hold and wait, no preemption, and circular wait. In the prevention version, both threads acquire resources in the same fixed order, eliminating the circular-wait condition.

RESULT

A multithreaded deadlock scenario and its prevention using resource ordering were successfully demonstrated.